Data-Driven Problem Solving

The Cooper Union for the Advancement of Science and Art

ME 371 - Fall 2025

Masoud Masoumi

Agenda

1. **Introduction to Linear Regression**
2. **Simple Linear Regression**
   - Concept & Implementation
3. **Multiple Linear Regression**
   - Concept & Vector notation
   - Implementation
4. **Evaluation Metrics**
   - MAE, MSE, RMSE, RAE, RSE, R^2
5. **Univariate & Multivariate Polynomial Regression**
   - Feature transformation & Implementation
6. **The Fitting Problem**

Linear Regression: Introduction

Linear regression is a bread-and-butter modeling technique that should serve as your baseline approach to building data-driven models. These models are typically:

- Easy to build
- Straightforward to interpret
- Often do quite well in practice

Two main applications:

- **Simplification & Compression**: See the trend and highlight the location and magnitude of outliers
- **Prediction & Forecasting**: Use the model to predict future or unknown values

Linear Regression: Mathematical Formulation

Linear Regression seeks the line $y = f(\mathbf{x})$ which minimizes the sum of the squared errors over all points, i.e. the coefficient vector $\mathbf{w}$ that minimizes:

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^{m} \left(y^{(i)} - \hat{y}^{(i)} \right)^2$$

with $f(\mathbf{x}) = \omega_0 + \sum_{i=1}^{n-1} \omega_i x_i$

where $\hat{y}^{(i)}$ is the estimated value at $\mathbf{x}^{(i)}$ and m is the number of samples.

Types of Linear Regression

Linear Regression can be divided into two types:

1. **Simple Linear Regression** (one independent variable)

$$f(\mathbf{x}) = \omega_0 + \omega_1 x_1$$

2. **Multiple Linear Regression** (multiple independent variables)

$$f(\mathbf{x}) = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + \ldots + \omega_{n-1} x_{n-1}$$

Simple Linear Regression

Simple Linear Regression: Finding Coefficients

For simple linear regression $f(\mathbf{x}) = \omega_0 + \omega_1 x_1$, we can find these coefficients using least squares method: https://www.real-statistics.com/regression/least-squares-method/least-squares-method-detailed/

$$\omega_1 = \frac{\sum_{i=1}^{m}(x^{(i)} - \bar{x})(y^{(i)} - \bar{y})}{\sum_{i=1}^{m}(x^{(i)} - \bar{x})^2}$$

$$\omega_0 = \bar{y} - \omega_1 \bar{x}_1$$

In practice, we use `scikit-learn` library which handles these calculations efficiently.

Simple Linear Regression: Implementation Steps

Step 1: Import required functions

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

Step 2: Define features (I.V.) and target variable (D.V.)

```
x_data = np.array(df[['independent variable']])
y_data = np.array(df[['dependent variable']])
```

Step 3: Split data into train and test sets

```
x_train, x_test, y_train, y_test = train_test_split(
    x_data, y_data, test_size=0.2, shuffle=True)
```

Simple Linear Regression: Implementation (cont.)

Step 4: Create linear regression object

```
lm = LinearRegression()
```

Step 5: Fit the model to training data

```
lm.fit(x_train, y_train)
```

Step 6: Make predictions using test data

```
yhat = lm.predict(x_test)
```

Step 7: Access the coefficients

```
w0 = lm.intercept_[0]     # ω₀
w1 = lm.coef_[0][0]       # ω₁
```

Model Evaluation: Metrics

- **Mean Absolute Error (MAE) =** $\frac{1}{m}\sum_{i=1}^{m}|y^{(i)}-\hat{y}^{(i)}|$
- **Mean Squared Error (MSE) =** $\frac{1}{m}\sum_{i=1}^{m}\left(y^{(i)}-\hat{y}^{(i)}\right)^2$
- **Root Mean Squared Error (RMSE) =** $\sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(y^{(i)}-\hat{y}^{(i)}\right)^2}$
- **Relative Absolute Error (RAE) =** $\frac{\sum_{i=1}^{m}|y^{(i)}-\hat{y}^{(i)}|}{\sum_{i=1}^{m}|y^{(i)}-\bar{y}|}$
- **Relative Squared Error (RSE) =** $\frac{\sum_{i=1}^{m}\left(y^{(i)}-\hat{y}^{(i)}\right)^2}{\sum_{i=1}^{m}\left(y^{(i)}-\bar{y}\right)^2}$
- **R-squared (R^2)** = 1 - RSE

Model Evaluation: Implementation

Mean Squared Error (MSE)

```
from sklearn.metrics import mean_squared_error
MSE = mean_squared_error(y_test, yhat)
```

Mean Absolute Error (MAE)

```
from sklearn.metrics import mean_absolute_error
MAE = mean_absolute_error(y_test, yhat)
```

R^2 Score

```
from sklearn.metrics import r2_score
r2score = r2_score(y_test, yhat)
```

Note: R^2 represents how close the data are to the fitted regression line. The higher the R^2, the better the model fits your data.

Multiple Linear Regression

Multiple Linear Regression: Formula

Multiple linear regression uses **multiple independent variables** to make a prediction.

We seek to find the best values for ω's in:

$$f(\mathbf{x}) = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + \dots$$

Using **vector notation**:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$$

where:

- $\mathbf{w}^T = [\omega_0, \omega_1, \omega_2, \omega_3, \dots]$ (coefficient vector)
- $\mathbf{x} = [1, x_1, x_2, x_3, \dots]^T$ (feature vector with bias term)

Multiple Linear Regression: Finding the Coefficients

These coefficients (weights) can be found using two main approaches:

1. Analytical Solution (Closed-Form)

- Solving the model parameters using closed-form equations
- Exact solution when computationally feasible
- See: Least Squares Method

2. Optimization Algorithm

- Gradient Descent and variants
- Iterative approach that minimizes the cost function
- Better for large datasets
- See: Gradient Descent Tutorial

NOTE: For a thorough mathematical explanation, see "Data-driven science and engineering: Machine learning, dynamical systems, and control" - Part II - Chapter 4.

Multiple Linear Regression: Implementation

The process is similar to simple linear regression, but with multiple features.

Step 1: Define features (multiple columns) and target variable

```
x_data = np.array(df[['independent variable 1', 'independent variable 2']]) # features/independent variables
y_data = np.array(df[['dependent variable']]) # target/dependent variable
```

Step 2: Split data and create model

```
x_train, x_test, y_train, y_test = train_test_split(
    x_data, y_data, test_size=0.2, shuffle=True)
lm = LinearRegression()
```

Step 3: Fit and predict

```
lm.fit(x_train, y_train)
yhat = lm.predict(x_test)
```

Multiple Linear Regression: Accessing Coefficients

For a model with multiple features, access coefficients as follows:

```
w0 = lm.intercept_[0]        # ω₀ (intercept)
w1 = lm.coef_[0, 0]          # ω₁ (first feature)
w2 = lm.coef_[0, 1]          # ω₂ (second feature)
w3 = lm.coef_[0, 2]          # ω₃ (third feature)
# ... and so on
```

Example output:

```
w_0 = 5.23, w_1 = 45.67, w_2 = -0.12, w_3 = 0.08
```

This means: $f(\mathbf{x}) = 5.23 + 45.67x_1 - 0.12x_2 + 0.08x_3$

Model Comparison Example

When comparing models with different numbers of features:

```
# Compare metrics across models
MAEs = [MAE_simple, MAE_multi1, MAE_multi2]
MSEs = [MSE_simple, MSE_multi1, MSE_multi2]
R2s = [r2_simple, r2_multi1, r2_multi2]
```

Key considerations:

- Do the predicted values make sense?
- Visualization of fit
- Evaluation metrics (MAE, MSE, R^2)
- Compare multiple models

Note: More features doesn't always mean better performance! Simple models often outperform complex ones.

Univariate Polynomial Regression

Uni. Polynomial Regression: Introduction

Polynomial Regression extends linear regression by allowing non-linear relationships between features and target variables.

A polynomial of degree n can be expressed as:

$$y = \omega_0 + \omega_1 x + \omega_2 x^2 + \omega_3 x^3 + \ldots + \omega_n x^n$$

Key Insight: By transforming features, we convert polynomial regression into a multiple linear regression problem!

We re-define the features as:

$$\begin{cases} x & = x_1 \\ x^2 & = x_2 \\ x^3 & = x_3 \\ \ldots & \end{cases}$$

so the polynomial becomes a multiple linear regression.

$$y = \omega_o + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + \ldots$$

Uni. Polynomial Regression: Feature Transformation

if we take a specific column in our data set as feature x, here is what this transformation will do for an n^{th} degree polynomial:

$$\begin{bmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(m)} \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & x^{(1)} & (x^{(1)})^2 & \dots & (x^{(1)})^n \\ 1 & x^{(2)} & (x^{(2)})^2 & \dots & (x^{(2)})^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x^{(m)} & (x^{(m)})^2 & \dots & (x^{(m)})^n \end{bmatrix}$$

This becomes a standard MLR: $y = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + \dots$

where $x_1 = x$, $x_2 = x^2$, $x_3 = x^3$, etc.

Uni. Polynomial Regression: Implementation Steps

Step 1: Import required functions including `PolynomialFeatures`

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import numpy as np
```

Step 2: Define features (independent variables) and target variable (dependent variable)

```
x_data = np.array(df[['independent variable']])
y_data = np.array(df[['dependent variable']])
```

Step 3: Split data into train and test sets

```
x_train, x_test, y_train, y_test = train_test_split(
    x_data, y_data, test_size=0.2, random_state=125)
```

Uni. Polynomial Regression: Implementation (cont.)

Step 4: Transform data using `PolynomialFeatures`

```
poly = PolynomialFeatures(degree=2)  # 2nd degree polynomial
x_train_transformed = poly.fit_transform(x_train)
x_test_transformed = poly.fit_transform(x_test)
```

Step 5: Create and fit the model

```
lm_poly = LinearRegression()
lm_poly.fit(x_train_transformed, y_train)
```

Step 6: Make predictions

```
yhat_test = lm_poly.predict(x_test_transformed)
yhat_train = lm_poly.predict(x_train_transformed)
```

Uni. Polynomial Regression: Accessing Coefficients

For a 2nd degree polynomial: $y = \omega_0 + \omega_1 x + \omega_2 x^2$

```
w0 = lm_poly.intercept_[0]        # ω₀
w1 = lm_poly.coef_[0][1]          # ω₁
w2 = lm_poly.coef_[0][2]          # ω₂

print(f'w_0 = {w0}, w_1 = {w1}, w_2 = {w2}')
```

For higher degree polynomials (e.g., 4th degree):

```
poly4 = PolynomialFeatures(degree=4)
x_train_transformed4 = poly4.fit_transform(x_train)
lm_poly4 = LinearRegression()
lm_poly4.fit(x_train_transformed4, y_train)

# Access coefficients: w0, w1, w2, w3, w4
w0 = lm_poly4.intercept_[0]
w_coefficients = lm_poly4.coef_[0][1:]  # All polynomial coefficients
```

Multivariate Polynomial Regression

Multivariate Polynomial Regression: Introduction

Multivariate Polynomial Regression extends polynomial regression to multiple features with polynomial terms AND interaction terms.

Univariate (one feature):

$$y = \omega_0 + \omega_1 x + \omega_2 x^2 + \omega_3 x^3$$

Multivariate (two features, degree 2):

$$y = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_1^2 + \omega_4 x_1 x_2 + \omega_5 x_2^2$$

Note the $x_1 x_2$ term - this is an **interaction term** capturing the combined effect of both features.

Key Insight: `PolynomialFeatures` automatically creates both polynomial terms and all interaction terms!

Multivariate Polynomial Regression: Feature Transformation

For two features and degree 2, the transformation creates:

$$\begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ x_1^{(2)} & x_2^{(2)} \\ \vdots & \vdots \\ x_1^{(m)} & x_2^{(m)} \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & (x_1^{(1)})^2 & x_1^{(1)}x_2^{(1)} & (x_2^{(1)})^2 \\ 1 & x_1^{(2)} & x_2^{(2)} & (x_1^{(2)})^2 & x_1^{(2)}x_2^{(2)} & (x_2^{(2)})^2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^{(m)} & x_2^{(m)} & (x_1^{(m)})^2 & x_1^{(m)}x_2^{(m)} & (x_2^{(m)})^2 \end{bmatrix}$$

General pattern: For k features and degree d, you get all combinations of terms where the sum of exponents ≤ d.

For 3 features, degree 2: $[1, x_1, x_2, x_3, x_1^2, x_1x_2, x_1x_3, x_2^2, x_2x_3, x_3^2]$

Multivariate Polynomial Regression: Implementation

```
# Step 1: Import (same as univariate)
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Step 2: Define MULTIPLE features (key difference!)
x_data = np.array(df[['ind. variable 1', 'ind. variable 2', 'ind. variable 3']])
y_data = np.array(df[['dependent variable']])

# Step 3-6: Same as univariate
x_train, x_test, y_train, y_test = train_test_split(x_data, y_data, test_size=0.2)

poly = PolynomialFeatures(degree=2)  # Creates all terms + interactions
x_train_transformed = poly.fit_transform(x_train)
x_test_transformed = poly.fit_transform(x_test)

lm_poly = LinearRegression()
lm_poly.fit(x_train_transformed, y_train)
yhat = lm_poly.predict(x_test_transformed)

# View created features
print(poly.get_feature_names_out())
```

Regression: Summary

Simple Linear Regression (one feature)

$$y = \omega_0 + \omega_1 x_1$$

Multiple Linear Regression (multiple features)

$$y = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \ldots + \omega_{n-1} x_{n-1} = \mathbf{w}^T \mathbf{x}$$

Univariate Polynomial Regression (one feature, polynomial terms)

$$y = \omega_0 + \omega_1 x + \omega_2 x^2 + \ldots + \omega_n x^n$$

Multivariate Polynomial Regression (multiple features, polynomial + interaction terms)

$$y = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_1^2 + \omega_4 x_1 x_2 + \omega_5 x_2^2 + \ldots$$

Objective: Minimize the cost function

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^{m} \left(y^{(i)} - \hat{y}^{(i)} \right)^2$$

where $\hat{y}^{(i)} = f(\mathbf{x}^{(i)})$ is the predicted value and m is the number of samples.

The Fitting Problem (Model Capacity)

The Fitting Problem: Generalization

The central challenge in machine learning: Our algorithm must perform well on new, previously unseen inputs - not just those on which our model was trained.

Generalization: The ability to perform well on previously unobserved inputs.

Two key factors determining model performance:

1. **Optimization**: Make the training error small
2. **Generalization**: Make the gap between training and test error small

These correspond to two central challenges: **underfitting** and **overfitting**.

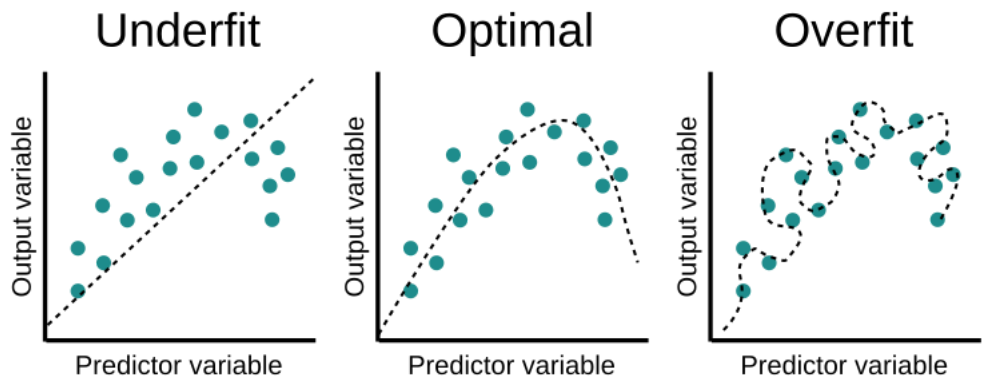
The Fitting Problem: Underfitting vs Overfitting

Underfitting

- Model is too simple to capture the underlying pattern
- Both training and test errors are high
- Poor performance on training and test data

Overfitting

- Model memorizes the training data rather than learning patterns
- Training error is very low, but test error is high
- Model fails to generalize to new data
- Often occurs when model is too complex



(source for image)

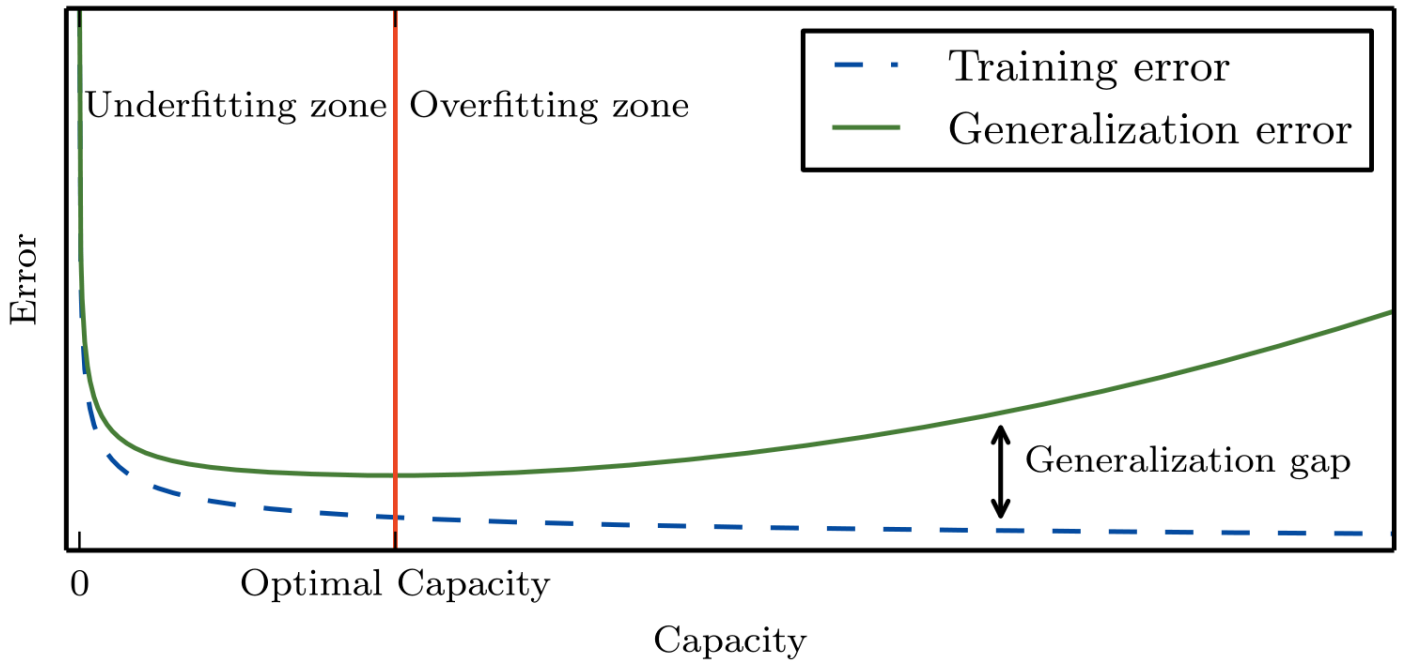
The Fitting Problem: Visual Understanding

Underfit Model (Low Capacity)

- Model is too simple with high bias, low variance

Overfit Model (High Capacity)

- Unable to generalize to unseen data with low bias, high variance (source for image)



Goal: Find the right balance - a model that captures the true pattern without memorizing noise.

Model Capacity and the Bias-Variance Tradeoff

Model Capacity: A model's ability to fit a wide variety of functions.

- **Low capacity models**: May struggle to fit training set (underfitting)
- **High capacity models**: May memorize training properties that don't generalize (overfitting)

Controlling capacity in polynomial regression:

- Polynomial degree controls model capacity
- Lower degree (1-2): Lower capacity, simpler model
- Higher degree (5+): Higher capacity, more complex model

Practical approach:

1. Train models with different polynomial degrees
2. Compare training and test errors
3. Select degree that balances both errors
4. Watch for divergence between training and test performance